



Database Systems

Project

Presented to:

Professor Asif Sohail

Presented by:

Habib Ghulam Bheek (BITF23M020)

Asim Iftikhar (BITF23M029)

Mohsin Ali (BITF23M031)

Database Systems - Project Report

1.Introduction to our project:

This database schema is designed to manage the operations and information flow of a hostel management system. It provides a structured approach to handle allottees, employees, inventory, departments, mess services, and accommodation. The schema ensures data integrity, establishes meaningful relationships between tables, and supports operations such as reporting, querying, and maintaining data consistency.

The database consists of the following key components:

- The ALLOTTEE table is the core of the system, storing detailed information about the hostel residents, including personal details like name, phone no, email, CNIC, date of birth, and address, as well as hostel-specific details like room assignment (ROOM_ID, BLOCK_ID), department (DEPTNO), and joining date.
- The ROOM and BLOCK tables manage accommodation in the hostel.
- The MESS_ENROLLMENT and MESS_FEE tables handle mess-related operations.
- The HOSTEL_EMPLOYEE table stores details about hostel employees, including their roles (JOB), salaries, and management hierarchy (MGR).
- The INVENTORY_ITEM table tracks all items assigned to rooms and blocks.
- The DEPARTMENT table records the academic departments linked to the hostel.
- The FEE table tracks all payments made by allottees for various services, including mess fees.

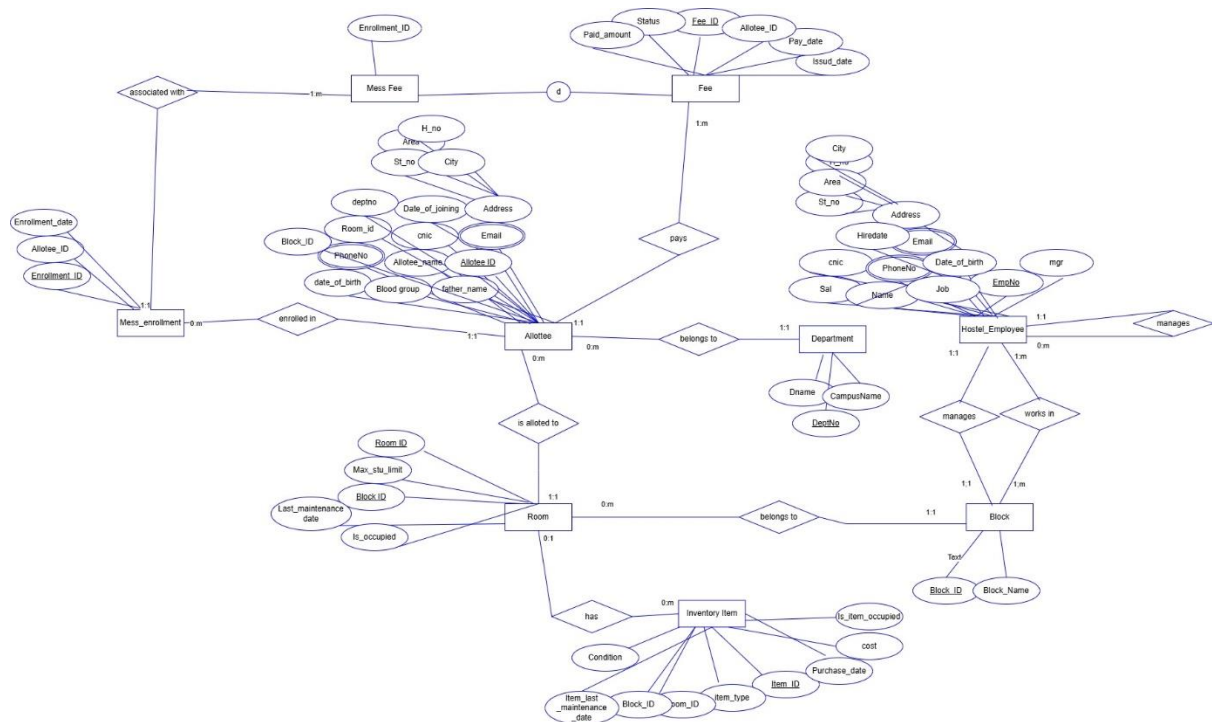
General Assumptions

- Each hostel is divided into multiple blocks, and each block contains multiple rooms.
- Each room has a maximum capacity (MAX_STU_LIMIT) and can house a limited number of allottees at any given time.
- Each allottee, employee, department, room, block, inventory item, fee, and enrollment record has a unique identifier (ALLOTTEE_ID, EMP_NO, DEPTNO, ROOM_ID, BLOCK_ID, ITEM_ID, FEE_ID, ENROLLMENT_ID).
- Each allottee can be assigned to one room and one block at a time.
- An allottee belongs to only one department but can have multiple contact details such as phone numbers and emails.
- Employees can have one or more contact details (phone numbers and emails).
- Each block is managed by a single employee (block manager), and a single employee can manage multiple blocks.
- An allottee must be enrolled in the mess to incur mess fees.
- Mess enrollment is tracked using the MESS_ENROLLMENT table, linked to allottees
- Fees have two possible statuses: Paid or Unpaid.
- A default status of Unpaid is set when a fee record is created.
- All mess fees are linked to the FEE table for payment tracking.

- Deletion of an allottee automatically deletes related mess enrollment and fee records (ON DELETE CASCADE).
- Each inventory item is assigned to a specific room and block.
- Inventory items are tracked for their condition (Good, Needs repairment, Unreparable) and occupancy status (Yes, No).
- Inventory and rooms have fields to track the last maintenance date for better management of resources.

2. ERD of the system

The entity relationship diagram of our project is as follows:



3. Construction of the Relational Schema by using both bottom-up approach and top-down approach. The schema must clearly mention the relational keys.

Top-down approach:

Allottee:

Allottee(**Allottee_ID**, Allottee_name, father_name, date_of_birth, cnic, Area, St_no, H_no, City, Date_of_joining, Blood_group, deptno, Room_ID, Block_ID)

Allottee Email:

Allotee_Email(**Allotee_ID, Email**)

Allotee_PhoneNo:

Allotee_PhoneNo(**Allotee_ID, PhoneNo**)

Mess_enrollment:

Mess_enrollment(**Enrollment_ID**, Enrollment_date, Allotee_ID)

Fee:

Fee(**Fee_ID**, Issued_date, Pay_date, Paid_amount, Status, Allotee_ID)

Mess_Fee:

Mess_Fee(**Fee_ID**, Enrollment_ID)

Room:

Room(**Room_ID, Block_ID** Max_stu_limit, Last_maintenance_date, Is_occupied)

Inventory_Item:

Inventory_Item(**Item_ID**, Room_ID, Block_ID, Item_type, Cost, Condition, Item_last_maintenance_date, Is_Item_Occupied, Purchase_date,)

Department

Department(**DeptNo**, Dname, CampusName)

Hostel_Employee:

Hostel_Employee(**Emp_No**, Name, Date_of_birth, cnic, mgr, Area, St_no, H_no, City, Hiredate, Sal, job)

Employee_Email:

Employee_Email(**Employee_ID, Email**)

Employee_PhoneNo:

Employee_PhoneNo(**Employee_ID, PhoneNo**)

Block:

Block(**Block_ID**, Block_Name, Block_mgr)

Block_Employee:

Block_Employee(**Block_ID, EmpNo**)

Now applying normalization as a validation technique:

Relations are in 1NF as there is no multivalued attribute or repeating group.

Relations are in 2NF as there is no partial dependency.

Relations are in 3NF as there is no transitive dependency.

Relations are in BCNF as there is no determinant that is not a candidate key.

Relations are in 4NF as there is no multivalued dependency.

Bottom-up approach:

Let's gather all the attributes in one relation:

R(Allotee_ID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic, Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining, Allotee_Blood_group, Deptno, Room_ID, Allotee_Email, Allotee_PhoneNo, Enrollment_ID, Enrollment_date, **Fee_ID**, Issued_date, Pay_date, Paid_amount, Status, Max_stu_limit, Last_maintenance_date, Is_occupied, **Item_ID**, Item_type, Cost, Condition, Item_last_maintenance_date, Is_Item_Occupied, Purchase_date,, Block_ID, Block_Name, mgr, **Emp_No**, Emp_Name, Emp_DOB, Emp_cnic, Emp_mgr, Emp_Area, Emp_St_no, Emp_H_no, Emp_City, Emp_Hiredate, Emp_Sal, Emp_Job, block_mgr, block_name, Dname, CampusName, Emp_email, Emp_phoneNo)

Fee_ID, item_ID, EmpNo can collectively serve as Composite PK for this relation and can uniquely identify a record

1NF:

In first normal form, we make sure that each row- column intersection holds atomic value.

Here multivalued attributes are:

Allotee_Email, Allotee_PhoneNo, Emp_email, Emp_phoneNo

We will have the following relations:

R1(Allotee_ID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic, Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining, Allotee_Blood_group, Deptno, Room_ID, Enrollment_ID, Enrollment_date, **Fee_ID**, Issued_date, Pay_date, Paid_amount, Status, Max_stu_limit, Last_maintenance_date, Is_occupied, **Item_ID**, Item_type, Cost, Condition, Item_last_maintenance_date, Is_Item_Occupied, Purchase_date,, Block_ID, Block_Name, mgr, **Emp_No**, Emp_Name, Emp_DOB, Emp_cnic, Emp_mgr, Emp_Area, Emp_St_no, Emp_H_no, Emp_City, Emp_Hiredate, Emp_Sal, Emp_Job, block_mgr, Dname, CampusName)

R2(**Fee_ID, Item_ID, Emp_No, Allotee_Email**)

R3(**Fee_ID, Item_ID, Emp_No, Allotee_PhoneNo**)

R4(**Fee_ID, Item_ID, Emp_No, Emp_email**)

R5(**Fee_ID, Item_ID, Emp_No, Emp_phoneNo**)

2NF

Let's look for partial dependencies

R2, R3, R4, R5 don't have any partial dependencies.

R1 have the following partial dependencies:

Fee_ID -> Allotee_ID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic, Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining, Allotee_Blood_group, Deptno, Room_ID, Enrollment_ID, Enrollment_date, Issued_date, Pay_date, Paid_amount, Status, Max_stu_limit, Last_maintenance_date, Is_occupied, Block_ID, Block_Name, Block_mgr, Dname, CampusName

Item_ID-> Item_type, Cost, Condition, Item_last_maintenance_date,
Is_Item_Occupied,Purchase_date,, Room_ID, Max_stu_limit,Last_maintenance_date,Is_occupied
Block_ID, Block_Name, block_mgr

Emp_No -> mgr, Emp_Name, Emp_DOB, Emp_cnic, Emp_mgr, Emp_Area, Emp_St_no, Emp_H_no,
Emp_City, Emp_Hiredate, Emp_Sal, Emp_Job

We will have the following relations in 2NF:

R1(Fee_ID, Item_ID, Emp_No)

R2(Fee_ID, Item_ID, Emp_No, Allotee_Email)

R3(Fee_ID, Item_ID, Emp_No, Allotee_PhoneNo)

R4(Fee_ID, Item_ID, Emp_No, Emp_email)

R5(Fee_ID, Item_ID, Emp_No, Emp_phoneNo)

R6(Fee_ID, Allotee_ID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic,
Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining,
Allotee_Blood_group, Deptno, Room_ID, Enrollment_ID, Enrollment_date, Issued_date, Pay_date,
Paid_amount, Status, Max_stu_limit, Last_maintenance_date, Is_occupied, Block_ID, Block_Name,
block_mgr, Dname, CampusName)

R7(Item_ID, Item_type, Cost, Condition, Item_last_maintenance_date,
Is_Item_Occupied, Purchase_date, Room_ID, Max_stu_limit, Last_maintenance_date, Is_occupied
Block_ID, Block_Name, block_mgr)

R8(Emp_No, mgr, Emp_Name, Emp_DOB, Emp_cnic, Emp_mgr, Emp_Area, Emp_St_no, Emp_H_no,
Emp_City, Emp_Hiredate, Emp_Sal, Emp_Job)

3NF:

Let's look for transitive dependencies:

R1 to R5 relations have no non-key attribute so they are already in 3NF,

In R6, we have the following transitive dependencies

Allotee_ID-> Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic,
Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining,
Allotee_Blood_group, Deptno, Room_ID, Max_stu_limit, Last_maintenance_date, Is_occupied,
Block_ID, Block_Name, block_mgr, Dname, CampusName

Enrollment_ID-> Enrollment_date, AlloteeID, Allotee_name, Allotee_father_name,
Allotee_date_of_birth, Allotee_cnic, Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City,
Allotee_Date_of_joining, Allotee_Blood_group, Deptno, Room_ID,
Max_stu_limit, Last_maintenance_date, Is_occupied, Block_ID, Block_Name, block_mgr, Dname,
CampusName

We will get the relations:

->R6(Fee_ID, Allotee_ID, Enrollment_ID, Issued_date, Pay_date, Paid_amount, Status)

R9(Allotee_ID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic,
Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining,

Allotee_Blood_group, Deptno, Room_ID, Max_stu_limit, Last_maintenance_date, Is_occupied , Block_ID, Block_Name, block_mgr, Dname, CampusName)

R10(**Enrollment_ID**, Enrollment_date, AlloteeID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic, Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining, Allotee_Blood_group, Deptno, Room_ID, Max_stu_limit, Last_maintenance_date, Is_occupied , Block_ID, Block_Name, block_mgr, Dname, CampusName)

In R10 we have the following transitive dependencies:

Allotee_ID → Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic, Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining, Allotee_Blood_group, Deptno, Room_ID, Max_stu_limit, Last_maintenance_date, Is_occupied , Block_ID, Block_Name, block_mgr, Dname, CampusName

It will decompose to R7 and:

→ R10(**Enrollment_ID**, Enrollment_date, AlloteeID)

In R9 we have the following transitive dependencies:

Deptno → Dname, CampusName

Block_ID, Room_ID → Max_stu_limit, Last_maintenance_date, Is_occupied , Block_Name, block_mgr

It will decompose to the following relations:

→ R9(**Allotee_ID**, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic, Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining, Allotee_Blood_group, Deptno, Room_ID, Block_ID)

→ R11(**Deptno**, Dname, CampusName)

R12(**Block_ID, Room_ID**, Max_stu_limit, Last_maintenance_date, Is_occupied , Block_Name, block_mgr)

R12 has a partial dependency:

Block_ID → Block_Name, block_mgr

R12 will give

→ R12(**Block_ID, Room_ID**, Max_stu_limit, Last_maintenance_date, Is_occupied)

→ R13(**Block_ID**, Block_Name, block_mgr)

In R7 we have following transitive dependencies:

Room_ID, Block_ID → Max_stu_limit, Last_maintenance_date, Is_occupied , , Block_Name, block_mgr

So it will give R12 , R13 and:

→ R7(**Item_ID**, Item_type, Cost, Condition, Item_last_maintenance_date, Is_Item_Occupied, Purchase_date,, Room_ID, Block_ID)

R8 have no transitive dependency.

In 3NF we will have the following relations:

R1(Fee ID, Item ID, Emp No)

R2(Fee ID, Item ID, Emp No, Allotee Email)

R3(Fee ID, Item ID, Emp No, Allotee PhoneNo)

R4(Fee ID, Item ID, Emp No, Emp_email)

R5(Fee ID, Item ID, Emp No, Emp_phoneNo)

R6(Fee ID, Enrollment_ID, Allotee_ID, Issued_date, Pay_date, Paid_amount, Status)

R7(Item ID, Item_type, Cost, Condition, Item_last_maintenance_date,
Is_Item_Occupied, Purchase_date,, Room_ID, Block_ID)

R8(Emp No, mgr, Emp_Name, Emp_DOB, Emp_cnic, Emp_mgr, Emp_Area, Emp_St_no, Emp_H_no,
Emp_City, Emp_Hiredate, Emp_Sal, Emp_Job)

R9(Allotee ID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic,
Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining,
Allotee_Blood_group, Deptno, Room_ID, Block_ID)

R10(Enrollment ID, Enrollment_date, AlloteeID)

R11(Deptno, Dname, CampusName)

R12(Block ID, Room ID, Max_stu_limit, Last_maintenance_date, Is_occupied)

R13(Block ID, Block_Name, block_mgr)

BCNF

All relations are already in BCNF

4NF

There is no multivalued dependency, so relations are already in 4NF.

After normalization we have the relations:

Fee_Item_Employee(Fee ID, Item ID, Emp No)

Allotee_Email(Fee ID, Item ID, Emp No, Allotee Email)

Allotee_PhoneNo(Fee ID, Item ID, Emp No, Allotee PhoneNo)

Employee_Email(Fee ID, Item ID, Emp No, Emp_email)

Employee_PhoneNo(Fee ID, Item ID, Emp No, Emp_phoneNo)

Allotee(Allotee ID, Allotee_name, Allotee_father_name, Allotee_date_of_birth, Allotee_cnic,
Allotee_Area, Allotee_St_no, Allotee_H_no, Allotee_City, Allotee_Date_of_joining,
Allotee_Blood_group, Deptno, Room_ID, Block_ID)

Inventory_Item(Item ID, Item_type, Cost, Condition, Item_last_maintenance_date,
Is_Item_Occupied, Purchase_date,, Room_ID, Block_ID)

Fee(Fee ID, Enrollment_ID, Allotee_ID, Issued_date, Pay_date, Paid_amount, Status)

Enrollment(**Enrollment_ID**, Enrollment_date, AlloteeID)

Employee(**Emp_No**, mgr, Emp_Name, Emp_DOB, Emp_cnic, Emp_mgr, Emp_Area, Emp_St_no, Emp_H_no, Emp_City, Emp_Hiredate, Emp_Sal, Emp_Job)

Department(**DeptNo**, Dname, CampusName)

Room(**Room_ID, Block_ID**, Max_stu_limit, Last_maintenance_date, Is_occupied)

Block(**Block_ID**, Block_Name, block_mgr)

4. Description of the relations in the following format:

Table Name: Allotee

Attribute	Data Type	Size	Constraints
Allotee_ID	Number	(4,0)	Primary Key
Allotee_name	Varchar2	30	Not Null
father_name	Varchar2	30	Nullable
date_of_birth	Date	7	Not Null
Cnic	Char	15	Not Null, Unique
Area	Varchar2	30	Nullable
St_no	Varchar2	10	Nullable
H_no	Varchar2	10	Nullable
City	Varchar2	20	Nullable
Date_of_joining	Date	7	Default Sysdate and Not Null
Blood_group	Varchar2	5	Nullable
Deptno	Number	(3,0)	Foreign Key (references Department(DeptNo))
Room_ID	Number	(4,0)	Foreign Key (references Room(Room_ID)), Part of composite FK
Block_ID	Number	(2,0)	Foreign Key (references Room(Block_ID)), Part of composite FK

Table Name: Allotee_Email

Attribute	Data Type	Size	Constraints
Allotee_ID	Number	(4,0)	Foreign Key (references Allotee(Allotee_ID)), Part of Primary key
Email	Varchar2	40	Not Null, Unique, Part of Primary key

Table Name: Allotee_PhoneNo

Attribute	Data Type	Size	Constraints
Allotee_ID	Number	(4,0)	Foreign Key (references Allotee(Allotee_ID)), Part of Primary key
PhoneNo	Number	(11,0)	Not Null, Unique, Part of Primary key

Table Name: Mess_enrollment

Attribute	Data Type	Size	Constraints
Enrollment_ID	Number	(4,0)	Primary Key
Enrollment_date	Date	7	Not Null Default Sysdate
Allotee_ID	Number	(4,0)	Foreign Key (references Allotee(Allotee_ID))

Table Name: Fee

Attribute	Data Type	Size	Constraints
Fee_ID	Number	(7,0)	Primary Key
Issued_date	Date	7	Default Sysdate and Not Null
Pay_date	Date	7	Nullable
Paid_amount	Number	(6,0)	Nullable
Status	Varchar2	6	Default Unpaid, Can be 'paid', 'unpaid', Not Null
Allotee_ID	Number	(4,0)	Foreign Key (references Allotee(Allotee_ID))

Table Name: Mess_Fee

Attribute	Data Type	Size	Constraints
Fee_ID	Number	(7,0)	Foreign Key (references Fee(Fee_ID)) Primary key
Enrollment_ID	Number	(4,0)	Foreign Key (references Mess_enrollment(Enrollment_ID))

Table Name: Room

Attribute	Data Type	Size	Constraints
Room_ID	Number	(4,0)	Part of Primary Key
Block_ID	Number	(2,0)	Foreign Key (references Block(Block_ID)), Part of Primary Key
Max_stu_limit	Number	(1,0)	Not Null
Is_occupied	Char	3	Default 'No', can be 'yes', 'No'
Last_maintenance_date	Date	7	

Table Name: Inventory_Item

Attribute	Data Type	Size	Constraints
Item_ID	Number	(7,0)	Primary Key
Room_ID	Number	(4,0)	Foreign Key (references Room(Room_ID)), Part of composite FK
Block_ID	Number	(2,0)	Foreign Key (references Room(Block_ID)), Part of composite FK
Cost	Number	(8,0)	Not Null
Condition	Varchar2	16	Default 'Good', can be 'Unrepairable', 'Needs repairment'
Item_last_maintenance_date	Date	7	Nullable
Is_Item_occupied	Varchar2	3	Default 'No', can be 'yes', 'No'
purchase_date	Date	7	Default Sysdate and Not Null
Item_type	Varchar2	30	Not Null

Table Name: Department

Attribute	Data Type	Size	Constraints
DeptNo	Number	(3,0)	Primary Key
Dname	Varchar2	30	Not Null
Campus_Name	Varchar2	30	Not Null

Table Name: Hostel_Employee

Attribute	Data Type	Size	Constraints
Emp_No	Number	(7,0)	Primary Key
Name	Varchar2	30	Not Null
Date_of_birth	Date	7	Not Null
Cnic	Char	15	Not Null, Unique
Mgr	Number	(7,0)	Foreign Key (references Hostel_Employees(Emp_No))
Area	Varchar2	30	Nullable
St_no	Varchar2	10	Nullable
H_no	Varchar2	10	Nullable
City	Varchar2	20	Nullable
Hiredate	Date	7	Default Sysdate, Not Null
Sal	Number	(8,0)	Not Null
Job	Varchar2	30	Not Null

Table Name: Employee_Email

Attribute	Data Type	Size	Constraints
Employee_ID	Number	(7,0)	Foreign Key (references Hostel_Employee(Emp_No)), Part of Primary key
Email	Varchar2	40	Part of Primary key

Table Name: Employee_PhoneNo

Attribute	Data Type	Size	Constraints
Employee_ID	Number	(7,0)	Foreign Key (references Hostel_Employee(Emp_No)), Part of Primary key
PhoneNo	Number	(11,0)	Part of Primary key

Table Name: Block

Attribute	Data Type	Size	Constraints
Block_ID	Number	(2,0)	Primary Key
Block_Name	Varchar2	30	Not Null
Block_Mgr	Number	(7,0)	Foreign Key (references Hostel_Employee(Emp_No))

Table Name: Employee_Block

Attribute	Data Type	Size	Constraints
Block_ID	Number	(2,0)	Foreign Key (references Block(Block_ID)), Part of Primary key
Emp_No	Number	(7,0)	Foreign Key (references Hostel_Employee(Emp_No)), Part of Primary key

5. CREATE TABLE statements for all the relations of your system.

Table Name: Mess_Enrollment

CREATE TABLE MESS_ENROLLMENT

(

ENROLLMENT_ID NUMBER(4,0) CONSTRAINT ENROLLMENT_PK PRIMARY KEY,

ENROLLMENT_DATE DATE DEFAULT SYSDATE NOT NULL,

ALLOTEE_ID CONSTRAINT ENROLLMENT_FK REFERENCES ALLOTEE(ALLOTEE_ID) ON DELETE CASCADE)

Table Name: Mess_Fee

```
CREATE TABLE MESS_FEE
(
FEE_ID NUMBER(7,0) CONSTRAINT MESS_PK REFERENCES FEE(FEE_ID),
ENROLLMENT_ID CONSTRAINT MESS_FEE_FK REFERENCES MESS_ENROLLMENT(ENROLLMENT_ID))
```

Table Name: Fee

```
CREATE TABLE FEE
(
FEE_ID NUMBER(7,0) CONSTRAINT FEE_PK PRIMARY KEY,
ISSUED_DATE DATE DEFAULT SYSDATE NOT NULL ,
PAYDATE DATE,
PAID_AMOUNT NUMBER(6,0) ,
STATUS VARCHAR2(6) DEFAULT 'Unpaid', ALLOTEE_ID CONSTRAINT FEE_FK REFERENCES
ALLOTEE(ALLOTEE_ID) ,
CONSTRAINT CHECK_STATUS CHECK(STATUS IN ('Paid','Unpaid')) )
```

Table Name: Allotee_Phone

```
CREATE TABLE ALLOTEE_PHONENO
(
ALLOTEE_ID CONSTRAINT ALLOTEE_PHONENO_FK REFERENCES ALLOTEE(ALLOTEE_ID) ON DELETE
CASCADE,
PHONENO VARCHAR2(15),
CONSTRAINT ALLOTEE_PHONENO_PK PRIMARY KEY(PHONENO, ALLOTEE_ID))
```

Table Name: Allotee_Email

```
CREATE TABLE ALLOTEE_EMAIL
(
ALLOTEE_ID CONSTRAINT ALLOTEE_EMAIL_FK REFERENCES ALLOTEE(ALLOTEE_ID) ON DELETE
CASCADE,
EMAIL VARCHAR2(30),
CONSTRAINT ALLOTEE_EMAIL_PK PRIMARY KEY(EMAIL, ALLOTEE_ID)
)
```

Table Name: Allottee

```
CREATE TABLE ALLOTTEE
(
  ALLOTTEE_ID NUMBER(4,0) CONSTRAINT ALLOTTEE_PK PRIMARY KEY,
  ALLOTTEE_NAME VARCHAR2(30) NOT NULL,
  FATHER_NAME VARCHAR2(30),
  DATE_OF_BIRTH DATE NOT NULL,
  CNIC CHAR(15) NOT NULL ,
  AREA VARCHAR2(30),
  ST_NO VARCHAR2(10),
  H_NO VARCHAR2(10),
  CITY VARCHAR2(20),
  DATE_OF_JOINING DATE DEFAULT SYSDATE NOT NULL ,
  BLOOD_GROUP VARCHAR2(5),
  DEPTNO NUMBER(3,0) CONSTRAINT ALLOTTEE_FK1 REFERENCES DEPARTMENT(DEPTNO) ON DELETE
  SET NULL,
  BLOCK_ID NUMBER(2,0),
  ROOM_ID NUMBER(4,0),
  CONSTRAINT ALLOTTEE_FK2 FOREIGN KEY(ROOM_ID, BLOCK_ID) REFERENCES ROOM(ROOM_ID,
  BLOCK_ID) ON DELETE SET NULL,
  CONSTRAINT ALLOTTEE_UNIQUE_CNIC UNIQUE(CNIC))
```

Table Name: Inventory_Item

```
CREATE TABLE INVENTORY_ITEM
(
  ITEM_ID NUMBER(7,0) CONSTRAINT ITEM_PK PRIMARY KEY,
  BLOCK_ID NUMBER(2,0),
  ROOM_ID NUMBER(4,0),
  COST NUMBER(8,0) NOT NULL,
  CONDITION VARCHAR2(16) DEFAULT 'Good',
  LAST_MAINTENANCE_DATE DATE,
  IS_ITEM_OCCUPIED VARCHAR2(3) DEFAULT 'No',
  PURCHASE_DATE DATE DEFAULT SYSDATE NOT NULL,
```

```
ITEM_TYPE VARCHAR2(30) NOT NULL,  
  
CONSTRAINT ITEM_FK FOREIGN KEY(ROOM_ID, BLOCK_ID) REFERENCES ROOM(ROOM_ID,  
BLOCK_ID) ON DELETE SET NULL,  
  
CONSTRAINT CHECK_CONDITION CHECK(CONDITION IN ('Good', 'Unrepairable','Needs repairment')),  
  
CONSTRAINT CHECK_OCCUPIED CHECK(IS_ITEM_OCCUPIED IN ('No', 'Yes')))
```

Table Name: Room

```
CREATE TABLE ROOM  
  
(  
  
ROOM_ID NUMBER(4,0),  
  
BLOCK_ID CONSTRAINT ROOM_FK REFERENCES BLOCK(BLOCK_ID),  
  
MAX_STU_LIMIT NUMBER(1,0) NOT NULL,  
  
IS_OCCUPIED CHAR(3) DEFAULT 'No',  
  
LAST_MAINTENANCE_DATE DATE,  
  
CONSTRAINT ROOM_OCCUPIED_CHECK CHECK(IS_OCCUPIED IN ('Yes','No') ),  
  
CONSTRAINT ROOM_PK PRIMARY KEY(ROOM_ID, BLOCK_ID)  
  
)
```

Table Name: Employee_Email

```
CREATE TABLE EMPLOYEE_EMAIL  
  
(  
  
EMP_NO CONSTRAINT EMP_EMAIL_FK REFERENCES HOSTEL_EMPLOYEE(EMP_NO) ON DELETE  
CASCADE,  
  
EMAIL VARCHAR2(30),  
  
CONSTRAINT EMP_EMAIL_PK PRIMARY KEY (EMAIL, EMP_NO))
```

Table Name: Employee_PhoneNo

```
CREATE TABLE EMPLOYEE_PHONENO  
  
(  
  
EMP_NO CONSTRAINT EMP_PNO_FK REFERENCES HOSTEL_EMPLOYEE(EMP_NO) ON DELETE  
CASCADE,  
  
PHONENO VARCHAR2(15) NOT NULL,  
  
CONSTRAINT EMP_PNO_PK PRIMARY KEY(PHONENO , EMP_NO)  
  
);
```

Table Name: Employee_Block

```
CREATE TABLE BLOCK_EMPLOYEE  
(  
BLOCK_ID CONSTRAINT EMP_BLOCK_FK1 REFERENCES BLOCK(BLOCK_ID),  
EMP_NO CONSTRAINT EMP_BLOCK_FK2 REFERENCES HOSTEL_EMPLOYEE(EMP_NO),  
CONSTRAINT BLOCK_EMP_PK PRIMARY KEY(BLOCK_ID, EMP_NO))
```

Table Name: Block

```
CREATE TABLE BLOCK  
(  
BLOCK_ID NUMBER (2,0) CONSTRAINT BLOCK_PK PRIMARY KEY,  
BLOCK_NAME VARCHAR2(30),  
BLOCK_MGR CONSTRAINT BLOCK_FK REFERENCES HOSTEL_EMPLOYEE(EMP_NO)  
)
```

Table Name: Department

```
CREATE TABLE DEPARTMENT  
(  
DEPTNO NUMBER (3,0) CONSTRAINT DEPT_PK PRIMARY KEY,  
DNAME VARCHAR2(30) NOT NULL,  
CAMPUS_NAME VARCHAR2(30) NOT NULL  
)
```

Table Name: Hostel_Employee

```
CREATE TABLE HOSTEL_EMPLOYEE  
(  
EMP_NO NUMBER (7,0) CONSTRAINT EMP_PK PRIMARY KEY,  
NAME VARCHAR2(30) NOT NULL,  
DATE_OF_BIRTH DATE NOT NULL,  
CNIC CHAR (15) NOT NULL,  
AREA VARCHAR2(30),  
ST_NO VARCHAR2(10),  
H_NO VARCHAR2(10),  
CITY VARCHAR2(20),
```



```
HIREDATE DATE DEFAULT SYSDATE NOT NULL,  
SAL NUMBER(8,0) NOT NULL,  
JOB VARCHAR2(30) NOT NULL,  
MGR NUMBER(7,0) CONSTRAINT EMP_FK REFERENCES HOSTEL_EMPLOYEE(EMP_NO),  
CONSTRAINT EMP_UNIQUE_CNIC UNIQUE(CNIC)  
);
```

6. Designing of at least two VIEWS, that you feel are the most important.

1. This view lists all mess enrollments with pending fees.

```
CREATE VIEW OUTSTANDING_MESS_FEES AS  
SELECT  
    ME.ENROLLMENT_ID,  
    ME.ALLOTEE_ID,  
    F.PAID_AMOUNT,  
    F.STATUS,  
    F.PAYDATE,  
    F.ISSUED_DATE  
FROM  
    MESS_ENROLLMENT ME  
JOIN MESS_FEE MF ON ME.ENROLLMENT_ID = MF.ENROLLMENT_ID  
JOIN FEE F ON MF.FEE_ID = F.FEE_ID  
WHERE  
    F.STATUS = 'Unpaid';
```

2. This view provides a count of allottees per department along with department details.

```
CREATE VIEW DEPARTMENT_ALLOTEES AS  
SELECT  
    D.DEPTNO,  
    D.DNAME,  
    D.CAMPUS_NAME,
```

```

COUNT(A.ALLOTTEE_ID) AS TOTAL_ALLOTEES
FROM
    DEPARTMENT D
LEFT JOIN ALLOTTEE A ON D.DEPTNO = A.DEPTNO
GROUP BY
    D.DEPTNO, D.DNAME, D.CAMPUS_NAME;

```

7. SELECT statement for at least five common reports to be generated by the system. The queries should include joins, subqueries, and other appropriate clauses as per requirement.

1. This query provide the total cost of inventory items per block and provides the count of items in each condition category.

```

SELECT
    I.BLOCK_ID,
    COUNT(CASE WHEN I.CONDITION = 'Good' THEN 1 END) AS GOOD_ITEMS,
    COUNT(CASE WHEN I.CONDITION = 'Needs repairment' THEN 1 END) AS REPAIR_NEEDED,
    COUNT(CASE WHEN I.CONDITION = 'Unrepairable' THEN 1 END) AS UNREPAIRABLE_ITEMS,
    SUM(I.COST) AS TOTAL_COST
FROM
    INVENTORY_ITEM I
GROUP BY
    I.BLOCK_ID
ORDER BY
    TOTAL_COST DESC;

```

2. This query lists the number of allottees in each block, grouped by block and room, along with the maximum capacity of each room.

```

SELECT
    B.BLOCK_NAME,
    R.ROOM_ID,
    R.MAX_STU_LIMIT,
    COUNT(A.ALLOTTEE_ID) AS CURRENT_OCCUPANTS

```

```
FROM
    BLOCK B
JOIN ROOM R ON B.BLOCK_ID = R.BLOCK_ID
LEFT JOIN ALLOTEE A ON R.ROOM_ID = A.ROOM_ID AND R.BLOCK_ID = A.BLOCK_ID
GROUP BY
    B.BLOCK_NAME, R.ROOM_ID, R.MAX_STU_LIMIT
HAVING
    COUNT(A.ALLOTEE_ID) > 0
ORDER BY
    B.BLOCK_NAME, R.ROOM_ID;
```

3. This query lists employees managing each block, along with their contact details.

```
SELECT
    B.BLOCK_ID,
    B.BLOCK_NAME,
    HE.EMP_NO,
    HE.NAME AS BLOCK_MANAGER,
    HE.CNIC,
    EE.EMAIL,
    EP.PHONENO
FROM
    BLOCK B
JOIN HOSTEL_EMPLOYEE HE ON B.BLOCK_MGR = HE.EMP_NO
LEFT JOIN EMPLOYEE_EMAIL EE ON HE.EMP_NO = EE.EMP_NO
LEFT JOIN EMPLOYEE_PHONENO EP ON HE.EMP_NO = EP.EMP_NO
ORDER BY
    B.BLOCK_ID;
```

4. This query provides the count of allottees grouped by their blood group and block.

```
SELECT
    A.BLOOD_GROUP,
    B.BLOCK_NAME,
    COUNT(A.ALLOTEE_ID) AS TOTAL_ALLOTEES
```

```
FROM  
  
    ALLOTEE A  
  
JOIN BLOCK B ON A.BLOCK_ID = B.BLOCK_ID  
  
GROUP BY  
  
    A.BLOOD_GROUP, B.BLOCK_NAME  
  
ORDER BY  
  
    A.BLOOD_GROUP, B.BLOCK_NAME;
```

5. This query provides the total salary expenditure per job role and lists the number of employees in each role.

```
SELECT  
  
    HE.JOB,  
  
    COUNT(HE.EMP_NO) AS TOTAL_EMPLOYEES,  
  
    SUM(HE.SAL) AS TOTAL_SALARY_EXPENDITURE,  
  
    ROUND(AVG(HE.SAL), 2) AS AVERAGE_SALARY  
  
FROM  
  
    HOSTEL_EMPLOYEE HE  
  
GROUP BY  
  
    HE.JOB  
  
ORDER BY  
  
    TOTAL_SALARY_EXPENDITURE DESC;
```

[PL/SQL: Implementation of at least one functions, one stored procedures, and one database triggers.](#)

Room Capacity Validation Triggers:

```
CREATE OR REPLACE TRIGGER INSERT_ROOM_CAPACITY BEFORE INSERT ON ALLOTEE  
  
FOR EACH ROW  
  
DECLARE  
  
    NoOfStdS NUMBER(2):=0;  
  
    MAX_CAP NUMBER(2);  
  
BEGIN  
  
    SELECT COUNT(ALLOTEE_ID) INTO NoOfStdS FROM ALLOTEE WHERE ROOM_ID = :NEW.ROOM_ID  
    AND BLOCK_ID = :NEW.BLOCK_ID;
```

```

SELECT MAX_STU_LIMIT INTO MAX_CAP FROM ROOM WHERE ROOM_ID = :NEW.ROOM_ID AND
BLOCK_ID = :NEW.BLOCK_ID;

IF (NoOfStds >= MAX_CAP)

THEN

raise_application_error(-20001,'The maximum occupancy limit for this room has been reached.');
```

END IF;

END;

```

CREATE OR REPLACE TRIGGER UPDATE_ROOM_CAPACITY BEFORE UPDATE ON ALLOTEE

FOR EACH ROW

DECLARE

NoOfStds NUMBER(2):=0;

MAX_CAP NUMBER(2);

BEGIN

SELECT COUNT(ALLOTEE_ID) INTO NoOfStds FROM ALLOTEE WHERE ROOM_ID = :NEW.ROOM_ID
AND BLOCK_ID = :NEW.BLOCK_ID;

SELECT MAX_STU_LIMIT INTO MAX_CAP FROM ROOM WHERE ROOM_ID = :NEW.ROOM_ID AND
BLOCK_ID = :NEW.BLOCK_ID;

IF (NoOfStds >= MAX_CAP)

THEN

raise_application_error(-20001,'The maximum occupancy limit for this room has been reached.');
```

END IF;

END;

Stored procedures:

Updating item condition

```

CREATE OR REPLACE PROCEDURE UPDATE_ITEM_CONDITION

( ITEM_NO NUMBER,

ITEM_CONDITION VARCHAR2)

IS

BEGIN

UPDATE INVENTORY_ITEM

SET CONDITION = ITEM_CONDITION WHERE ITEM_ID = ITEM_NO;
```

END;

Updating fee status

CREATE OR REPLACE PROCEDURE UPDATE_FEE_STATUS

(FEE_NO NUMBER,

FEE_STATUS VARCHAR2)

IS

BEGIN

UPDATE FEE

SET STATUS= FEE_STATUS WHERE FEE_ID = FEE_NO ;

END;

Functions:

Checking Availability status

CREATE OR REPLACE FUNCTION GET_AVAILABILITY_STATUS

(ROOM_NO NUMBER,

BLOCK_NO NUMBER)

RETURN VARCHAR2

IS

MAX_CAP NUMBER(2);

CURRENT_OCCUPANCY NUMBER(2);

BEGIN

SELECT MAX_STU_LIMIT INTO MAX_CAP FROM ROOM WHERE ROOM_ID = ROOM_NO AND
BLOCK_ID = BLOCK_NO;

SELECT COUNT(ALLOCATE_ID) INTO CURRENT_OCCUPANCY FROM ALLOCATE WHERE
ROOM_ID = ROOM_NO AND BLOCK_ID = BLOCK_NO ;

IF (CURRENT_OCCUPANCY >= MAX_CAP)

THEN

RETURN 'UNAVAILABLE';

ELSE

RETURN 'AVAILABLE';

END IF;

EXCEPTION

WHEN NO_DATA_FOUND THEN

RETURN 'NO SUCH ROOM EXIST';

END;